“EduConnect”

Learn Without Boundaries

Software Requirements Specification

Prepared by

Student ID	Name
23301130	Nishat Afrin Fariha
23301136	Moumita Muzammal
23301556	Nafisa Nawar Adrita

1. Introduction

1.1 Type of Project

EduConnect is a comprehensive, full-stack Educational Web Application and Learning Management System (LMS) designed to facilitate a streamlined interface between academic seekers and expert educators.

1.2 Purpose

The purpose of EduConnect is to address the fragmentation currently present in the private tutoring and online learning markets. By providing a centralized digital ecosystem, EduConnect empowers students to seamlessly discover and enroll in high-quality video courses, engage in real-time interactive live classes, and leverage a localized Tuition Hub for personalized academic support. The system aims to bridge the gap between structured curriculum-based learning and flexible, on-demand private tutoring.

1.3 Scope

The system's scope encompasses the end-to-end design, development, and deployment of a responsive web application. The platform is engineered using the MERN stack (MongoDB, Express.js, React.js, Node.js) and strictly adheres to the Model-View-Controller (MVC) architectural pattern to ensure modularity and code reusability. The system provides role-specific functionality for:

- **Students:** Exploration of the course catalog, progress monitoring, tuition request management, and real-time interaction with tutors.
- **Verified Instructors:** Content creation, curriculum management, live session broadcasting, and monetization tracking.
- **Administrators:** Platform-wide moderation, instructor credential verification, and conflict resolution via content reporting mechanisms.

1.4 Definitions, Acronyms, and Abbreviations

- **MERN:** A JavaScript-based stack comprising MongoDB (Database), Express.js (Backend Framework), React.js (Frontend Library), and Node.js (Runtime).

- **MVC:** Model-View-Controller; a design pattern that separates application logic into three interconnected components to improve maintainability.
- **LMS:** Learning Management System; a software application for the administration, documentation, tracking, and delivery of educational courses.
- **VOD:** Video on Demand; a media distribution system that allows users to access video content without a traditional static broadcasting schedule.
- **API:** Application Programming Interface; the set of protocols used to enable communication between the frontend and backend or external services like Agora.io and SSL Commerz.

1.5 References

- Department of Computer Science and Engineering, BRAC University. *CSE470: Software Engineering, Project Outline (Spring '26 Onwards)*.
- React/Node.js/MongoDB Official Technical Documentation.
- IEEE Software Engineering Standards (IEEE 830-1998) for SRS documentation guidelines.

1.6 Overview

This Software Requirements Specification (SRS) is structured to provide a holistic view of the EduConnect development project. Section 2 describes the product perspective, target demographics, and operational constraints. Section 3 delineates the functional and non-functional requirements. Section 4 provides the structural framework for the system's architecture via the Class Diagram. Sections 5 and 6 detail the technology stack and system compatibility. Finally, Section 7 offers a visual implementation roadmap through placeholders for key features, concluding with an analysis of potential development challenges and overall project objectives.

2. Overall Description

2.1 Product Perspective

EduConnect is a self-contained, web-based Learning Management System (LMS) that acts as

an all-in-one educational hub. Unlike fragmented learning platforms, EduConnect provides an integrated ecosystem that bridges the gap between formal course-based learning and informal localized tutoring. The system is designed as an independent product that operates independently but interacts with third-party APIs to deliver specialized functionalities, such as real-time video streaming via Agora.io and secure financial transactions via the SSL Commerz Sandbox. It serves as a central data repository for educational content, tuition opportunities, and student-instructor interaction histories.

2.2 Product Features

The system is characterized by a high degree of role-based functionality. Key feature pillars include:

- **Monetized Learning:** A structured course management module with gating logic, progress tracking, and automated certification.
- **Real-Time Interaction:** A sophisticated live-class engine supporting scheduling, recording, and archive management.
- **Community Marketplace:** A token-based "Tuition Hub" that connects students with tutors based on proximity and specific subject requirements.
- **Moderation & Quality Assurance:** A multi-layered administrative moderation system involving instructor verification and content flagging.
- **Subscription Ecosystem:** A revenue-focused backend that governs publishing rights and feature usage limits based on dynamic instructor subscription tiers.

2.3 User Classes and Characteristics

- **Students:** Primary users who are typically non-technical. They require a seamless, intuitive UI to manage their tuition listings, course enrollments, and live class schedules. Their primary goal is efficient learning and academic support.
- **Verified Instructors:** Expert users who require a high-performance content studio. They are responsible for curriculum management, live teaching, and professional branding. They possess a higher level of familiarity with digital tools.
- **Administrators:** Super-users responsible for system integrity. They require access to administrative moderation dashboards to audit instructor verification requests, handle community reports, and manage platform-wide content queues.

2.4 Operating Environment

EduConnect is a responsive web application designed for platform-agnostic performance. The frontend is optimized for compatibility with all modern desktop and mobile browsers (e.g., Chrome, Firefox, Safari). The backend environment utilizes the Node.js runtime, ensuring a non-blocking, asynchronous execution model perfect for handling real-time features like live streaming and chat. Data persistence is managed by a cloud-ready MongoDB instance.

2.5 Constraints

- **Architectural Mandate:** The system MUST strictly adhere to the Model-View-Controller (MVC) architectural pattern to ensure code separation, modularity, and maintainability.
- **Development Cycle:** The software is to be delivered within a rigid 4-sprint Agile development cycle, requiring high code efficiency and minimal technical debt.
- **Security:** As a system handling financial transactions and personal academic data, the application must implement industry-standard authentication (JWT) and data validation.

2.6 Assumptions and Dependencies

- **Infrastructure:** It is assumed that the production environment will have high-speed bandwidth, essential for real-time Agora.io video broadcasting.
- **Third-Party Availability:** The system's functionality is dependent on the availability and stability of external third-party service providers (Agora, SSL Commerz).
- **User Competency:** It is assumed that end-users possess basic web literacy to interact with course materials and the tuition hub.

3.1 Functional Requirements (20 Features)

3.1.1 Instructor Verification & Onboarding

- Instructors submit academic/professional credentials via a multi-step form.
- The system immediately changes user status to "Pending Review" and notifies the Admin Dashboard.

3.1.2 Instructor Subscription Gateway

- The system applies a 1-month "Free Trial" status to newly verified instructors.
- After the trial, the system enforces a monthly payment barrier to maintain publishing access.

3.1.3 Instructor Content Studio

- A secure, role-based dashboard exclusively accessible to verified instructors.
- Displays real-time analytics on revenue, course enrollments, and student progress metrics.

3.1.4 Course Curriculum Builder

- An interface for instructors to structure lessons into modules.
- Includes toggle controls to mark specific lectures as "Preview/Unlocked" for non-enrolled students.

3.1.5 Admin Moderation Panel

- A master dashboard for administrators to view, audit, and approve/reject instructor applications.
- Includes a "Publishing Queue" to review new course content before it goes live.

3.1.6 Student Dashboard & Progress Tracking

- Monitors video viewing events via the React video player.
- Updates a dynamic database field that visualizes completion percentage on the student profile.

3.1.7 Assignment Evaluation Module

- Enables instructors to create assignment prompts with submission deadlines.
- Allows students to upload files, triggering an "Evaluation Needed" status for the instructor.

3.1.8 Automated Certificate Generation

- Triggers logic upon 100% video completion and instructor-graded assignment approval.
- Uses a library to render a PDF certificate with dynamic student data.

3.1.9 Live Broadcasting Integration

- Integrates Agora.io API to generate unique stream tokens for video rooms.
- Supports both ad-hoc live classes and scheduled session creation.

3.1.10 Subscription-Based Live Class Quotas

- The backend validates the instructor’s subscription tier against their monthly usage.
- Automatically disables the "Start Live" function if the current quota (e.g., 20 sessions) is met.

3.1.11 Student Notification & Reminder System

- Students can interact with a "Remind Me" bell icon on class cards.
- The system stores these preferences and triggers an in-app alert before the session starts.

3.1.12 Live Class Recording & VOD Archive

- Provides instructors an option to "Archive Recording" post-session.
- Automatically maps the video URL to a public "Archived Classes" directory upon approval.

3.1.13 Token-Based Tuition Hub Posting

- Checks the user’s wallet balance before allowing a post creation.
- Executes a database transaction to deduct one token upon successful submission.

3.1.14 Tuition Hub Dynamic Filtering

- Fetches filtered data from the database based on UI parameters (subject, salary, location).
- Uses efficient indexing to ensure fast search results on the tuition board.

3.1.15 Tutor Application Routing

- Captures applicant data and stores it in an "Applications Received" table.
- Notifies the original poster of the tuition listing via a dashboard indicator.

3.1.16 Real-Time In-App Messaging

- Utilizes Socket.io for bi-directional, real-time message delivery.
- Persists chat history in MongoDB, allowing retrieval upon session restart.

3.1.17 Course Review & Rating Engine

- Validates purchase history before allowing a user to submit a rating.
- Aggregates data to calculate an average star rating displayed on course cards.

3.1.18 Featured Instructor Algorithm

- Aggregates total courses, live class count, and average ratings to generate a score.
- Sorts and renders the "Featured Instructor" list on the home page.

3.1.19 Community Content Reporting

- Provides a "Report Content" modal on courses and live class interfaces.
- Routes reports to a dedicated admin queue for moderation and potential suspension.

3.1.20 Secure Payment Gateway Integration

- Handles checkout sessions using SSL Commerz Sandbox API keys.
- Processes success/failure callbacks to update user wallet or subscription status.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- **Response Time:** The system shall respond to user interactions (clicks, form submissions) within 2 seconds under normal network conditions.
- **Throughput:** The system shall be capable of handling at least 50 concurrent users per server instance without degrading the quality of live video streaming or real-time chat.
- **Latency:** Live video sessions hosted via Agora.io shall maintain an end-to-end latency of less than 400ms to ensure real-time interaction between instructors and students.

3.2.2 Security Requirements

- **Data Protection:** All sensitive user data, including passwords, shall be hashed using `bcrypt` with a minimum of 10 salt rounds before being stored in the MongoDB database.
- **Access Control:** The system shall implement JSON Web Tokens (JWT) for secure session management. Tokens shall be set to expire every 24 hours, necessitating re-authentication.
- **Input Validation:** The backend shall sanitize all user inputs to prevent NoSQL injection, Cross-Site Scripting (XSS), and CSRF attacks.

3.2.3 Reliability and Availability

- **System Availability:** The platform shall maintain 99% uptime during standard operational hours (08:00 – 23:00 BDT).
- **Fault Tolerance:** In the event of a database connection failure, the system shall provide user-friendly error messages rather than exposing raw stack traces.

- **Data Integrity:** The system shall ensure ACID compliance for financial transactions processed through the SSL Commerz gateway to prevent partial or duplicate payments.

3.2.4 Usability Requirements

- **Responsiveness:** The user interface shall be fully responsive, maintaining a consistent layout and functionality across mobile devices, tablets, and desktop browsers (Viewport width 320px to 1920px).
- **Accessibility:** The UI shall follow basic W3C accessibility guidelines, including sufficient color contrast and descriptive labels for all form inputs.
- **Learnability:** The platform shall be designed so that a first-time user can successfully navigate from the login page to course enrollment in under 3 minutes without external assistance.

3.2.5 Maintainability and Scalability

- **Modular Architecture:** The system must adhere strictly to the MVC design pattern, ensuring that the Model (Data), View (UI), and Controller (Logic) layers remain decoupled to facilitate easier updates and feature additions.
- **Scalability:** The backend services shall be stateless, allowing the platform to scale horizontally by deploying additional Node.js instances if user traffic increases beyond current capacity.

4. Class Diagram

[Will be added later]

5. Tools and Technologies

EduConnect is built using the **MERN stack**, ensuring a strictly decoupled **MVC architecture** where components are modular and scalable.

- **Frontend (View):** React.js is used to build a dynamic, responsive user interface, handling all client-side rendering and state management.

- **Backend (Controller):** Node.js provides the runtime environment, while Express.js acts as the primary controller, managing routing, HTTP requests, and business logic.
- **Database & ODM (Model):** MongoDB serves as the NoSQL database for flexible data storage, paired with Mongoose to enforce data schemas and integrity.
- **Integration Services:** The platform integrates Socket.io for real-time messaging, Agora.io for live video broadcasting, and the SSL Commerz Sandbox for secure payment testing.
- **Development Workflow:** The project uses Git for version control and Visual Studio Code for development and debugging.

6. Compatibility

EduConnect is designed as a platform-agnostic, responsive web application. It ensures consistent performance and accessibility across the following environments:

- **Browser Compatibility:** The application is fully compatible with all modern, standards-compliant web browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- **Device Responsiveness:** The user interface utilizes a fluid grid system, ensuring full functionality and optimized layout rendering across desktop monitors, laptops, tablets, and mobile smartphones.
- **Operating Systems:** As a web-based solution, the platform operates independently of the user's underlying operating system, requiring only a functional browser environment on Windows, macOS, Linux, Android, or iOS.
- **Server Environment:** The backend infrastructure is built on Node.js, ensuring compatibility with standard cloud hosting providers (e.g., AWS, Heroku, or Vercel) and containerized environments like Docker.

7. Implementation

[Will be added later]

8. Challenges

[Will be added later]

9. Conclusion

EduConnect successfully demonstrates the execution of a complex, full-stack educational platform adhering strictly to the MVC design pattern. By effectively integrating course management, real-time communication, and a localized tuition marketplace into exactly 20 core features, the project meets all academic requirements while providing a scalable solution for digital learning.

10. References React Documentation:

- React Documentation: https://react.dev/
- Express.js MVC Guidelines: https://expressjs.com/
- Mongoose Documentation: https://mongoosejs.com/
- Agora.io Video SDK Documentation: https://docs.agora.io/en/
- SSL Commerz Integration Guides: https://developer.sslcommerz.com/doc/v4/